

How to Install and Deploy Terraform on Ubuntu

1. Update System Packages:

\$ sudo apt update && sudo apt upgrade -y

```
md_rustam@DESKTOP-CPK0PUB:~$ sudo apt update && sudo apt upgrade -y
[sudo: authenticate] Password:
Hit:1 https://download.docker.com/linux/ubuntu resolute InRelease
Get:2 http://security.ubuntu.com/ubuntu resolute-security InRelease [137 kB]
Hit:3 http://archive.ubuntu.com/ubuntu resolute InRelease
Hit:4 https://ppa.launchpadcontent.net/ansible/ansible/ubuntu resolute InRelease
Get:5 http://archive.ubuntu.com/ubuntu resolute-updates InRelease [137 kB]
Get:6 http://security.ubuntu.com/ubuntu resolute-security/main amd64 Components [32.8 kB]
Get:7 http://security.ubuntu.com/ubuntu resolute-security/universe amd64 Components [43.1 kB]
Hit:8 http://archive.ubuntu.com/ubuntu resolute-backports InRelease
Get:9 http://archive.ubuntu.com/ubuntu resolute-updates/main amd64 Packages [446 kB]
Get:10 http://archive.ubuntu.com/ubuntu resolute-updates/main amd64 Components [84.9 kB]
Get:11 http://archive.ubuntu.com/ubuntu resolute-updates/universe amd64 Packages [235 kB]
Get:12 http://archive.ubuntu.com/ubuntu resolute-updates/universe amd64 Components [166 kB]
Get:13 http://archive.ubuntu.com/ubuntu resolute-updates/multiverse amd64 Packages [11.3 kB]
Fetched 1294 kB in 3s (460 kB/s)
All packages are up to date.
Get more security updates through Ubuntu Pro with 'esm-apps' enabled:
  libmagickcore-7.q16-10 imagemagick-7-common imagemagick python3-pip-whl
  libmagickwand-7.q16-10 libopenexr-3-1-30 libmagickcore-7.q16-10-extra
  imagemagick-7.q16 python3-pip
Learn more about Ubuntu Pro at https://ubuntu.com/pro
Summary:
  Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 0
```

2. Install Prerequisites:

\$ sudo apt install -y gnupg software-properties-common curl

```
md_rustam@DESKTOP-CPK0PUB:~$ sudo apt install -y gnupg software-properties-common curl
gnupg is already the newest version (2.4.8-4ubuntu3).
software-properties-common is already the newest version (0.120.1).
curl is already the newest version (8.18.0-1ubuntu2.3).
Summary:
  Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 0
```

3. Add HashiCorp's GPG key and repository:

```
$ wget -O- https://apt.releases.hashicorp.com/gpg | \
gpg --dearmor | \
$ sudo tee /usr/share/keyrings/hashicorp-archive-keyring.gpg > /dev/null
```

```
$ echo "deb [signed-by=/usr/share/keyrings/hashicorp-archive-keyring.gpg]
https://apt.releases.hashicorp.com $(lsb_release -cs) main" | \ sudo tee
/etc/apt/sources.list.d/hashicorp.list
```

```
md_rustam@DESKTOP-CPK0PUB:~$ wget -O- https://apt.releases.hashicorp.com/gpg | \
gpg --dearmor | \
sudo tee /usr/share/keyrings/hashicorp-archive-keyring.gpg > /dev/null

echo "deb [signed-by=/usr/share/keyrings/hashicorp-archive-keyring.gpg]
https://apt.releases.hashicorp.com $(lsb_release -cs) main" | \ sudo tee
/etc/apt/sources.list.d/hashicorp.list
--2026-08-09 09:34:02-- https://apt.releases.hashicorp.com/gpg
Resolving apt.releases.hashicorp.com (apt.releases.hashicorp.com)... 108.158.232.60, 108.158.232.79, 108.158.232.90,
Connecting to apt.releases.hashicorp.com (apt.releases.hashicorp.com)|108.158.232.60|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 3980 (3.9K) [binary/octet-stream]
Saving to: 'STDOUT'

-
100%[=====] 3.89K --.-KB/s in 0s

2026-08-09 09:34:02 (77.0 MB/s) - written to stdout [3980/3980]

Command 'sudo' not found, did you mean:
  command 'sudo' from deb sudo (1.9.17p2-1ubuntu3)
  command 'sudo' from deb sudo-rs (0.2.13-0ubuntu1)
Try: sudo apt install <deb name>
-bash: /etc/apt/sources.list.d/hashicorp.list: No such file or directory
```

4. Install Terraform:

```
$ sudo apt update  
$ sudo apt install terraform
```

5. Verify the installation:

```
$ terraform -version
```

```
md_rustam@DESKTOP-CPK0PUB:~$ sudo terraform -version  
[sudo: authenticate] Password:  
Terraform v1.9.5  
on linux_amd64  
  
Your version of Terraform is out of date! The latest version  
is 1.15.8. You can update by downloading from https://www.terraform.io/downloads.html
```

6. Set up a working directory and write config:

```
$ mkdir ~/terraform-project && cd ~/terraform-project  
$ vi main.tf
```

Add your provider and resource blocks (example for AWS):

```
terraform {  
  required_providers {  
    aws = {  
      source = "hashicorp/aws"  
      version = "~> 5.0"  
    }  
  }  
}  
  
provider "aws" {  
  region = "us-east-1"  
}  
  
resource "aws_instance" "example" {  
  ami      = "ami-0abcdef1234567890"  
  instance_type = "t2.micro"  
}
```

7. Initialize, plan, and apply:

\$ terraform init # downloads providers/modules

\$ terraform validate # checks syntax

```
md_rustam@DESKTOP-CPK0PUB:~/terraform-project$ terraform init
Initializing the backend...
Initializing provider plugins...
- Finding hashicorp/aws versions matching "~> 5.0"...
- Installing hashicorp/aws v5.100.0...
- Installed hashicorp/aws v5.100.0 (signed by HashiCorp)
Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.
```

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see any changes that are required for your infrastructure. All Terraform commands should now work.

If you ever set or change modules or backend configuration for Terraform, rerun this command to reinitialize your working directory. If you forget, other commands will detect it and remind you to do so if necessary.

\$ terraform plan # shows what will be created/changed

\$ terraform apply # applies the changes (type "yes" to confirm)

8. Single Terraform configuration that launches Linux EC2 instances in two AWS regions using provider aliases.

\$ vi main.tf

```
terraform {
  required_version = ">= 1.5.0"
  required_providers {
    aws = {
      source = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
}

# Providers for two regions
# -----
provider "aws" {
  alias = "region1"
  region = "us-east-1"
}

provider "aws" {
  alias = "region2"
  region = "eu-west-1"
}
```

```
# Latest Amazon Linux 2023 AMI - Region 1
```

```
data "aws_ami" "al2023_region1" {  
  provider = aws.region1  
  most_recent = true  
  owners = ["amazon"]
```

```
  filter {  
    name = "name"  
    values = ["al2023-ami-*-x86_64"]  
  }  
}
```

```
  filter {  
    name = "virtualization-type"  
    values = ["hvm"]  
  }  
}
```

```
# Latest Amazon Linux 2023 AMI - Region 2
```

```
data "aws_ami" "al2023_region2" {  
  provider = aws.region2  
  most_recent = true  
  owners = ["amazon"]
```

```
  filter {  
    name = "name"  
    values = ["al2023-ami-*-x86_64"]  
  }  
}
```

```
  filter {  
    name = "virtualization-type"  
    values = ["hvm"]  
  }  
}
```

```
# EC2 Instance - Region 1 (us-east-1)
```

```
resource "aws_instance" "instance_region1" {  
  provider = aws.region1  
  ami = data.aws_ami.al2023_region1.id  
  instance_type = var.instance_type
```

```
  tags = {  
    Name = "instance-us-east-1"  
  }  
}
```

```

# EC2 Instance - Region 2 (eu-west-1)

resource "aws_instance" "instance_region2" {
  provider    = aws.region2
  ami         = data.aws_ami.al2023_region2.id
  instance_type = var.instance_type

  tags = {
    Name = "instance-eu-west-1"
  }
}

# Variables

variable "instance_type" {
  description = "EC2 instance type"
  type        = string
  default     = "t2.micro"
}

# Outputs

output "region1_instance_id" {
  value = aws_instance.instance_region1.id
}

output "region1_public_ip" {
  value = aws_instance.instance_region1.public_ip
}

output "region2_instance_id" {
  value = aws_instance.instance_region2.id
}

output "region2_public_ip" {
  value = aws_instance.instance_region2.public_ip
}

```

9. *Deployment steps:*

- # 1. Set up AWS credentials (if not already configured)
aws configure
- # 2. Initialize Terraform (downloads AWS provider)
terraform init
- # 3. Validate syntax

terraform validate

4. Preview the changes

terraform plan

5. Apply — creates instances in both regions

terraform apply